Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

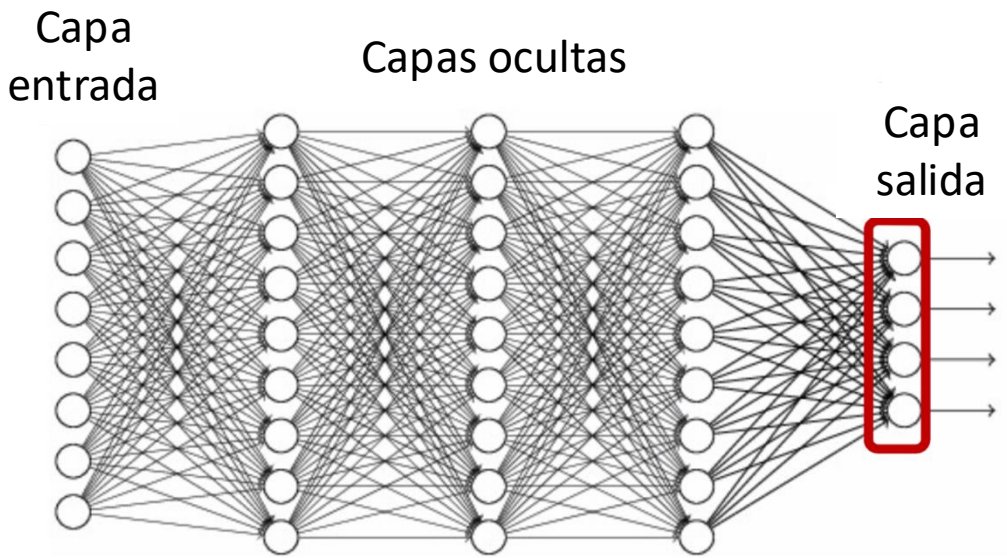
Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 11 - PAO 2026 – I

Agenda

1. Aprendizaje en las redes MLP
    - 1.1 Formas de entrada – salida
    - 1.2 Multi-clase representación one-hot
    - 1.3 Funciones de error y de umbral
    - 1.4 Aprendizaje con gradiente descendiente en las redes MLP
        - 1.4.1 Forward y back-propagation en las redes MLP
        - 1.4.2 Jacobian
2. Regularización en las redes MLP

Aprendizaje: La salida



- La salida generalmente es un valor real
  - Si la salida es escalar: Una neurona
  - Salida es un vector: Tantas neuronas como se requiera en la salida
  - $d = [d_1 d_2 \dots d_L]$

Aprendizaje: Salida multi-clase– representación one-hot

- Considere una red que debe distinguir si una entrada es: perro, mono, flor, sombrero o gato
- Podemos representar este conjunto con el siguiente vector:

  [perro mono flor sombrero gato]
- Para entradas de cada una de estas clases, la salida deseada es:

  perro: $[1\ 0\ 0\ 0\ 0]^T$

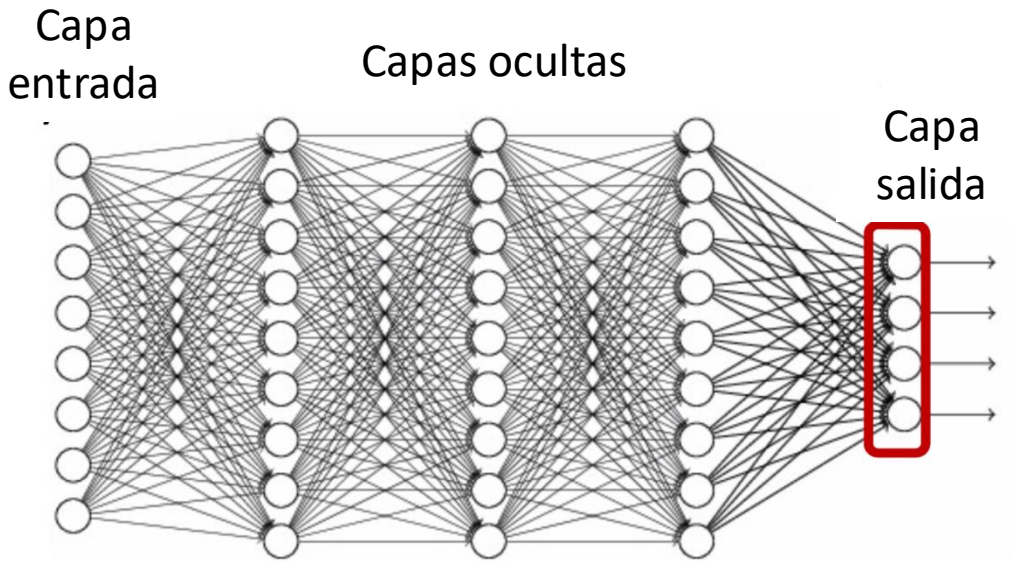
  mono: $[0\ 1\ 0\ 0\ 0]^T$

  flor: $[0\ 0\ 1\ 0\ 0]^T$

  sombrero: $[0\ 0\ 0\ 1\ 0]^T$

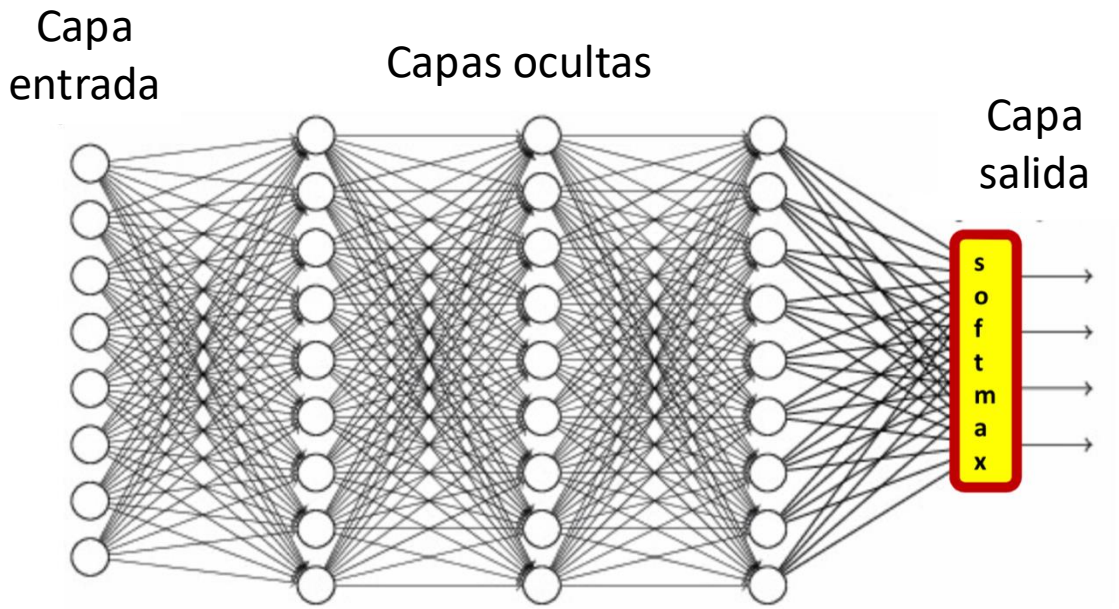
  gato: $[0\ 0\ 0\ 0\ 1]^T$
- Para una entrada de cualquier clase, tendremos un vector de salida de cinco dimensiones con cuatro ceros y un 1 en la posición de la clase correspondiente
- Este se conoce como one-hot encoding

Aprendizaje: Salida en las redes multi-clase



- Para un clasificador de clases múltiples con L clases, la representación one-hot tiene L salidas binarias
  - Un vector binario L-dimensional
- La salida de la red es binaria (L-1 ceros y un 1 en el lugar correcto)
- Si la salida es un vector de probabilidades, como en el caso de una función sigmoidal
  - L probabilidades que suman 1

Aprendizaje: Redes multi-clase - salida



- Es común utilizar la función de activación Softmax en la salida de las redes de clasificación multi-clase

$$z_i = \sum_j W_{ji}^{(n)} y_j^{(n-1)}; \qquad y_i = \frac{e^{(z_i)}}{\sum_j e^{(z_j)}}$$

- Esta salida puede interpretarse como la probabilidad $y_i = P(clase = i|X)$

Aprendizaje: Qué es $f()$

- Softmax
  - Las unidades Softmax se usan para distribuciones de salida Multi-clase, se usa para una distribución de probabilidad sobre una variable discreta con n valores posibles, como una generalización de la función Sigmoide que representa la distribución de probabilidad sobre una variable binaria:

$$\mathrm{softmax}(\boldsymbol{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

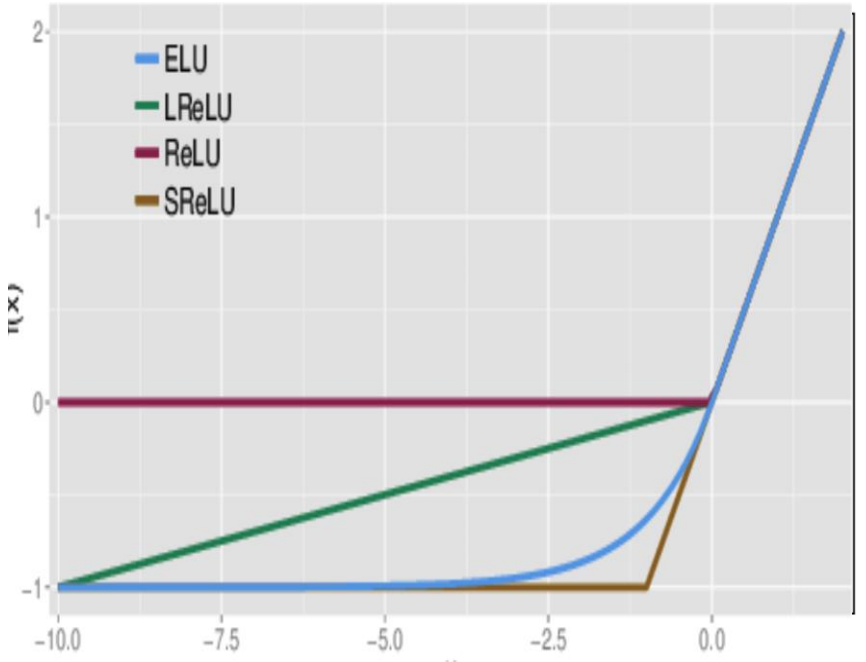
Aprendizaje: Qué es $f()$

Las opciones para unidades ocultas son:

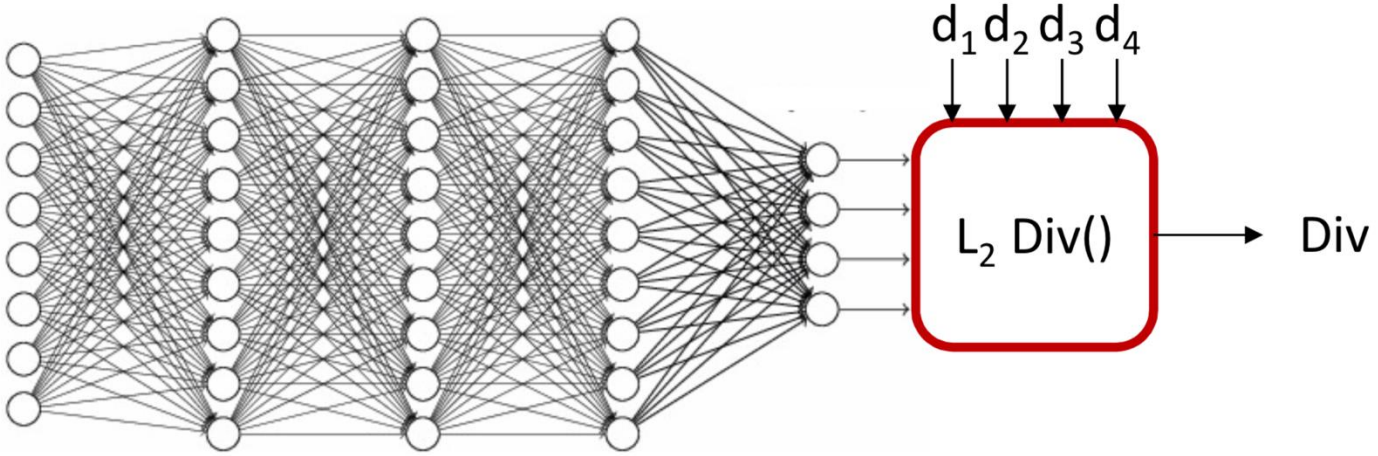
- Unidades lineales rectificadas (Rectified linear units – Relu)

$$g(z) = \max\{0, z\}$$

- Hay algunas generalizaciones de Relu:
  - Rectificación de valor absoluto
  - ReLU exponencial - ELU
  - ReLU con fugas – LReLU: Leaky ReLU
  - ReLU paramétrico – SReLU: Shifted ReLU



Aprendizaje: Ejemplos de funciones $div()$

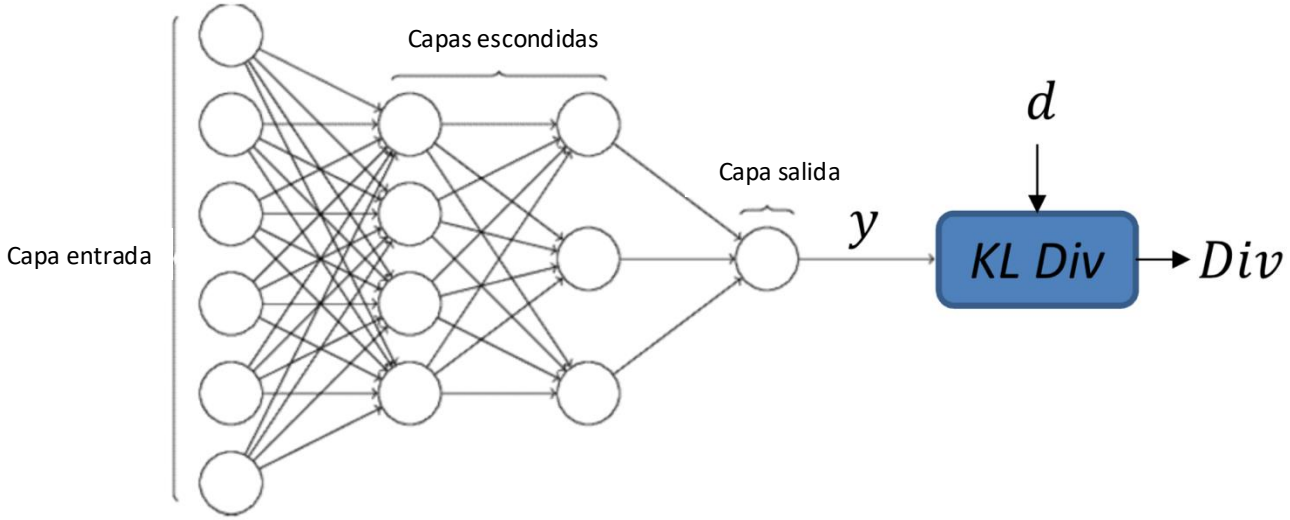


- El cuadrado de la distancia Euclideana entre la salida real y la deseada, conocida como la divergencia L_2 (es popular)

$$Div(Y, d) = \frac{1}{2}\|Y - d\|^2 = \frac{1}{2}\sum_i (y_i - d_i)^2$$

Aprendizaje: Ejemplos de funciones $div()$

- Clasificador binario

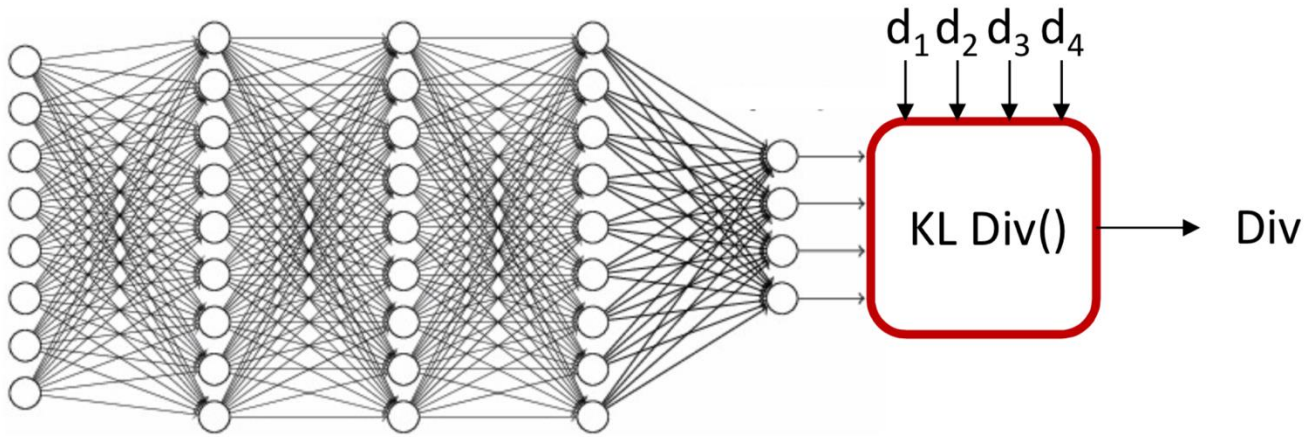


- La salida de un clasificador binario $Y \in [0,1], d\ es\ 0/1.$
- Una función común es el cálculo de la entropía (cross entropy) entre la distribución probabilística de la salida real $[Y, 1-Y]$ y la probabilidad de la salida deseada $[d, 1-d]$

$$Div(Y,d) = -dlogY - (1-d)log(1-Y)$$

Aprendizaje: Ejemplos de funciones $div()$

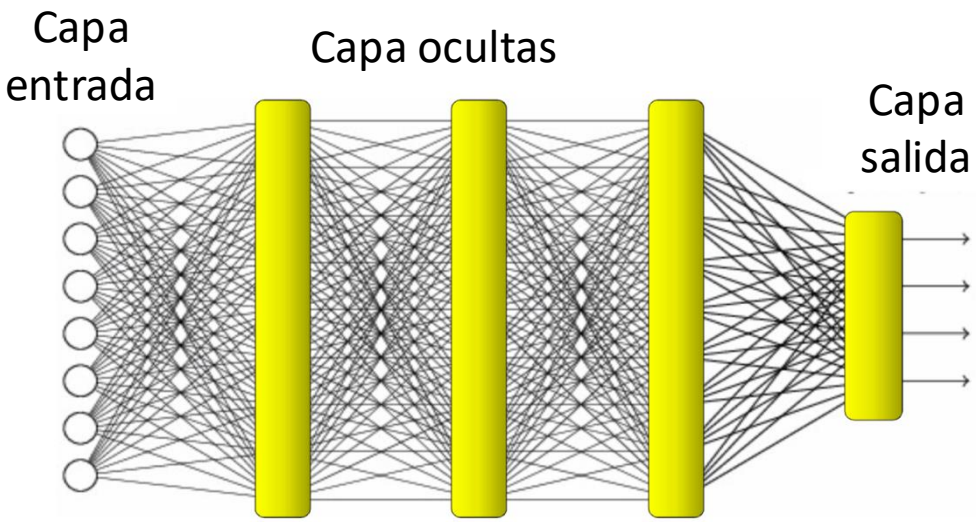
- Clasificador multi-clase



- La salida deseada es: un vector "one-hot" [00 ... 1 ... 000] con un 1 en la posición c^{th} (para clase c)
  - La salida es la distribución probabilística $[y_1, y_2, \ldots]$
  - La entropía cruzada entre la salida multiple deseada y la salida real:

$$Div(Y, d) = -\sum_i d_i log\ y_i$$

Aprendizaje: Arquitectura típica de una red



$$z_j^{(1)} = \sum_i w_{ij}^{(1)} x_i$$

$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_j^{(k-1)}$$

$$y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

- Cada capa de perceptrones se la puede considerar como un vector de activación

Aprendizaje: Entrenamiento con GD

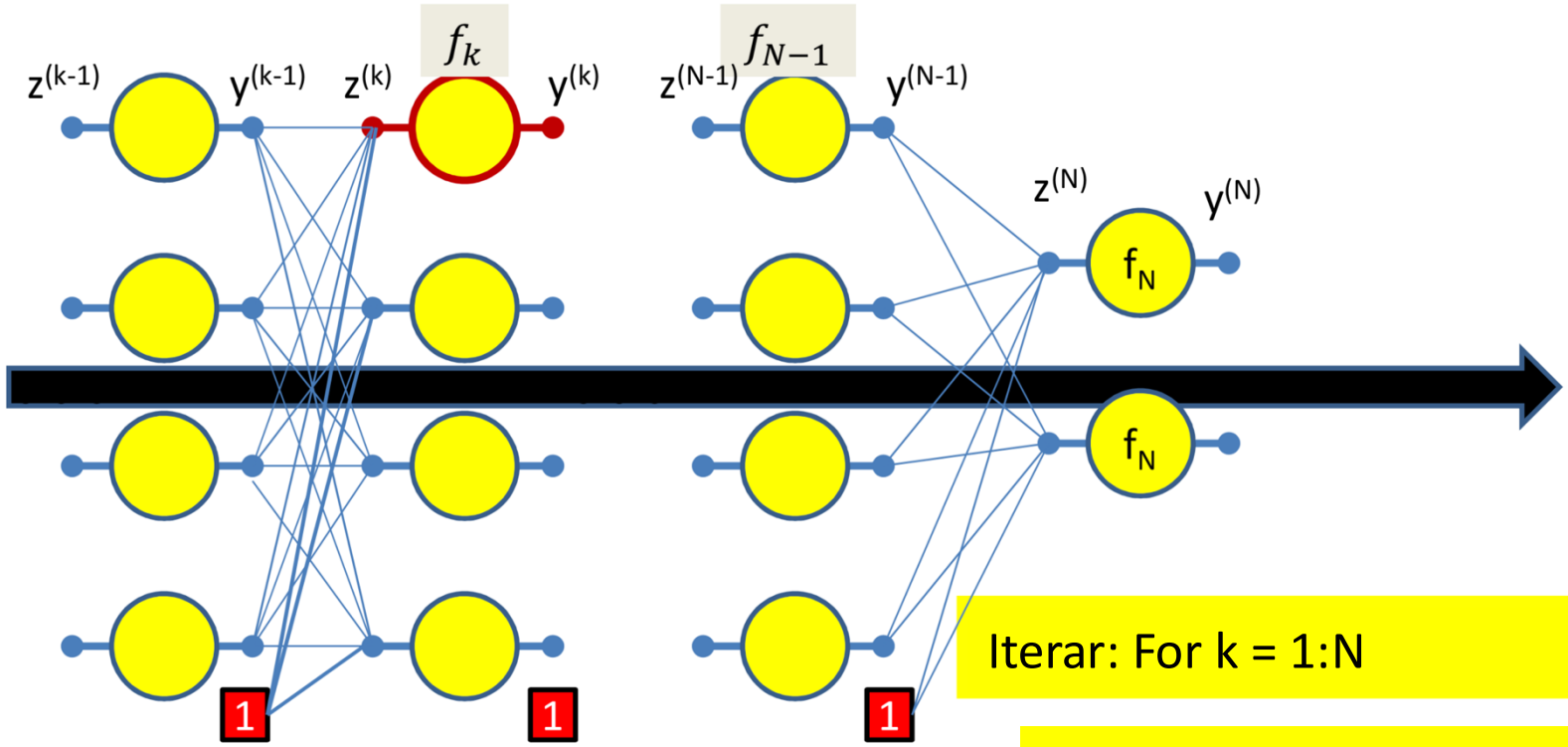
El error total en entrenamiento:

$$Error = \frac{1}{M}\sum_{m} Div(Y_m, d_m)$$

La derivada:

$$\frac{dError}{dW_{i,j}^{(k)}} = \frac{1}{M}\sum_{m} \frac{dDiv(Y_m, d_m)}{dW_{i,j}^{(k)}}$$

Aprendizaje: Forward



Iterar: For k = 1:N

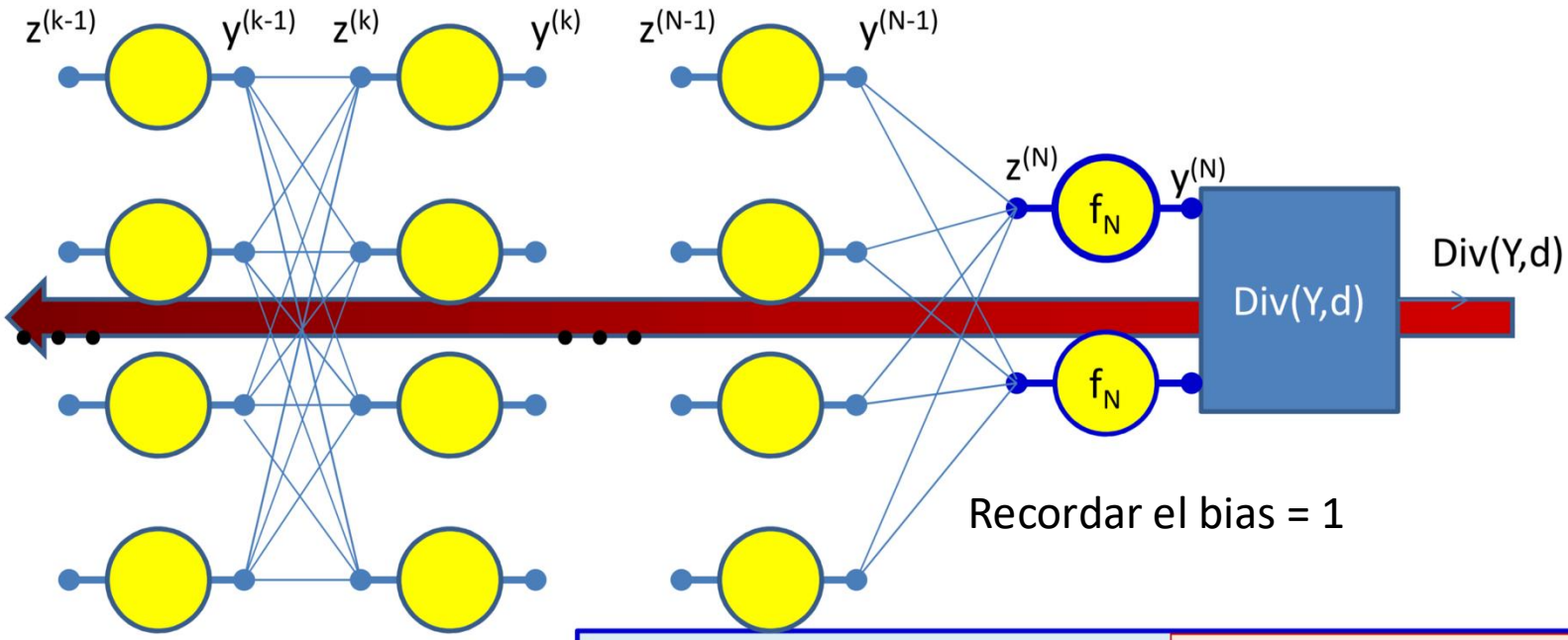
For i,j = 1:Ancho de la capa

$$y_i^{(0)} = x_i$$

$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_i^{(k-1)}$$

$$y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

Aprendizaje: Backward



Recordar el bias = 1

Inicializar: Gradiente c.r.a salida de la red

$$\frac{\partial Div}{\partial y_i} = \frac{\partial Div(Y,d)}{\partial y_i^{(N)}}$$

For k = N-1..1
For i, j = 1..Ancho de la capa

$$\frac{\partial Div}{\partial z_i^{(k)}} = f_k'\left(z_i^{(k)}\right)\frac{\partial Div}{\partial y_i^{(k)}}$$

$$\frac{\partial Div}{\partial y_i^{(k-1)}} = \sum_j w_{ij}^{(k)}\frac{\partial Div}{\partial z_j^{(k)}}$$

$$\frac{\partial Div}{\partial w_{ij}^{(k)}} = y_i^{(k-1)}\frac{\partial Div}{\partial z_j^{(k)}}$$

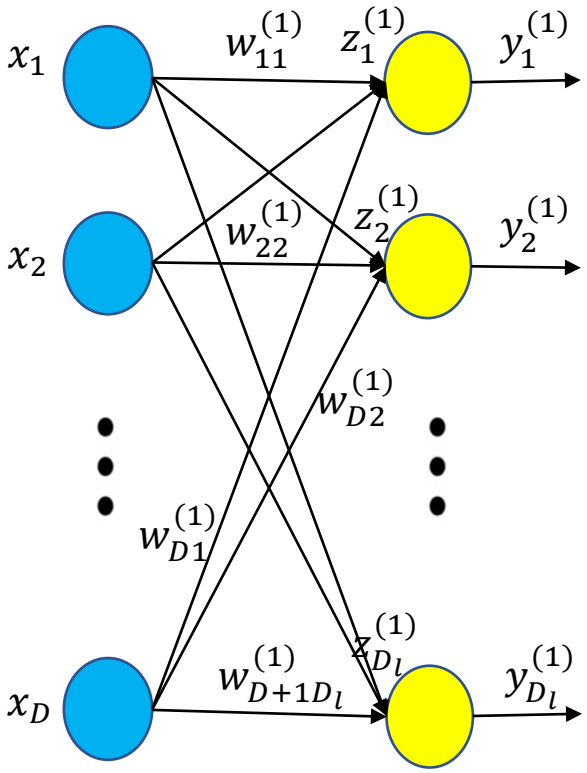
Aprendizaje: Entrenamiento con BP

- Inicializar todos los pesos $\{W^{(1)}, W^{(2)}, \ldots, W^{(K)}\}$
- Realizar:
  - Inicializar $Error = 0$, Para todas las i, j, k, inicializar $\frac{dError}{dW_{i,j}^{(k)}} = 0$
  - Para todos los ejemplos $m = 1: M$
    - Para cada capa k y para todas las unidades i, j:
      - Paso "Forward":
        - Calcular la salida Y_m
        - Calcular $Error \mathrel{+}= Div(Y_m, d_m)$
      - Paso "Backward":
        - Calcular: $\frac{dDiv(Y_m, d_m)}{dW_{i,j}^{(k)}}$
        - Calcular: $\frac{dError}{dW_{i,j}^{(k)}} \mathrel{+}= \frac{dDiv(Y_m, d_m)}{dW_{i,j}^{(k)}}$
  - Para todas las i, j, k actualizar:

$$W_{i,j}^{(k)} = W_{i,j}^{(k)} - \frac{\propto}{M} \frac{dError}{dW_{i,j}^{(k)}}$$

  - Hasta que el $Error$ converja

Aprendizaje: En Deep Learning – formulación vectorial



$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} \qquad Z_k = \begin{bmatrix} z_1^{(k)} \\ z_2^{(k)} \\ \vdots \\ z_{D_k}^{(k)} \end{bmatrix} \qquad y_k = \begin{bmatrix} y_1^{(k)} \\ y_2^{(k)} \\ \vdots \\ y_{D_k}^{(k)} \end{bmatrix}$$

$$W = \begin{bmatrix} w_{01}^{(1)} & w_{01}^{(2)} & \cdots & w_{01}^{(N)} \\ w_{11}^{(1)} & w_{11}^{(2)} & \cdots & w_{11}^{(N)} \\ \vdots & \vdots & & \vdots \\ w_{D_{l-1}D_l}^{(1)} & w_{D_{l-1}D_l}^{(2)} & \cdots & w_{D_{l-1}D_l}^{(N)} \end{bmatrix} \qquad b_k = \begin{bmatrix} b_1^{(k)} \\ b_2^{(k)} \\ \vdots \\ b_{D_l}^{(k)} \end{bmatrix}$$

- El cálculo en forma vectorial de una capa se expresa:

$$Z_k = W_k y_{k-1}$$
$$y_k = f_k(Z_k)$$

Aprendizaje: Formulación vectorial

- La derivada de una función vectorial c.r.a un vector de entrada se llama "Jacobian"
- Es una matriz de derivadas parciales, que tiene la forma:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = f\left(\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_D \end{bmatrix}\right)$$

En notación vectorial

$$\mathbf{y} = f(\mathbf{z})$$

$$J_{\mathbf{y}}(\mathbf{z}) = \begin{bmatrix} \frac{\partial y_1}{\partial z_1} & \frac{\partial y_1}{\partial z_2} & \cdots & \frac{\partial y_1}{\partial z_D} \\ \frac{\partial y_2}{\partial z_1} & \frac{\partial y_2}{\partial z_2} & \cdots & \frac{\partial y_2}{\partial z_D} \\ \cdots & \cdots & \ddots & \cdots \\ \frac{\partial y_M}{\partial z_1} & \frac{\partial y_M}{\partial z_2} & \cdots & \frac{\partial y_M}{\partial z_D} \end{bmatrix}$$

$$\Delta y = J_y(z)\Delta z$$

Regularización MLP: Dropout para manejar overfitting

Dropout

- A cada unidad de procesamiento de la RN (excepto las que están en la capa de salida) se le da una probabilidad *p* de ser temporalmente ignorada en los cálculos.
- El hiper-parámetro *p* se denomina tasa de dropout.
- En cada iteración, se selecciona aleatoriamente las neuronas que se dejan fuera (dropout), de acuerdo con la probabilidad asignada.
- Como resultado, cada vez trabajamos con una RN más pequeña.
- El dropout ignora unidades de procesamiento durante la fase de entrenamiento. Algunas unidades no se consideran durante el paso forward ni en backward.